

Comprehensive Analysis of 4th- and 6th-Order Summation-By-Parts Operators in Spherical Coordinates

Antigravity Computational Framework

Advanced Agentic Coding

(Dated: March 9, 2026)

This scientific report details the theoretical foundation, methodological construction, and programmatic implementation of 4th-order and 6th-order Summation-By-Parts (SBP) finite difference operators in Julia (`sbp4.jl` and `sbp6.jl`). These numerical operators strictly mimic the continuous integration-by-parts principle on discretely bounded domains, ensuring discrete energy stability for initial-boundary value problems (IBVPs) through dynamically solved rational linear constraint systems.

I. INTRODUCTION AND THEORETICAL CONTEXT

The Summation-By-Parts (SBP) framework is a discrete, systematic mechanism for constructing highly stable numerical schemes for partial differential equations (PDEs), such as initial-boundary value problems (IBVPs). By mirroring strict continuous functional properties analytically at the discrete stage, numerical stability flows naturally via the global energy method without resorting to ad-hoc, destabilizing numerical dissipation parameters.

A. The Continuous Integration-By-Parts Principle

In the continuous functional space, the generic 1D Integration-By-Parts (IBP) property for bounded, continuously differentiable functions $u(x)$ and $v(x)$ on a radial spatial domain $[0, R]$ is algebraically stated as:

$$\int_0^R u \frac{\partial v}{\partial x} dx = [u \cdot v]_0^R - \int_0^R \frac{\partial u}{\partial x} v dx \quad (1)$$

B. The Discrete SBP Form

An underlying finite difference first-derivative operator D satisfies the strict SBP property if it can be analytically factored into the universal form:

$$D = P^{-1}Q \quad (2)$$

where:

- P is a symmetric positive-definite (SPD) matrix defining the discrete volume norm (quadrature). The inner product is $(u, v)_P = u^T P v \approx \int u v dx$.
- Q is an “almost” skew-symmetric matrix such that its symmetric part extracts the exact physical boundary evaluations:

$$Q + Q^T = B = \text{diag}(-1, 0, \dots, 0, 1) \quad (3)$$

Evaluating the discrete derivative through the inner product yields $(u, Dv)_P + (Du, v)_P = u^T B v$, structurally confirming discrete energy conservation identical to the analytical IBP boundary condition model.

C. Diagonal vs. Block-Diagonal Norms and Boundary Accuracy

Traditionally, a perfectly diagonal norm P requires the interior difference stencil yielding $2s$ -order accuracy to drop steeply to s -order accuracy across boundary closures. If one upgrades P to an extended block-diagonal structure (where nodes strictly near the boundary interact non-diagonally), higher accuracy boundary closures can be preserved (commonly $2s - 1$).

The implementations analyzed herein (`sbp4.jl` and `sbp6.jl`) utilize an advanced *split-pattern formulation* primarily tailored toward spherical/radial coordinates incorporating r^p metric terms. Instead of a single unmanageable non-diagonal P , the system separates into a strictly diagonal scalar mass matrix S evaluated alongside a structured, split vector mass matrix V . The localized divergence becomes:

$$D = S^{-1}(B - G_{\text{even}}^T V) \quad (4)$$

This architecture retains peak memory sparsity everywhere except within the explicitly specified coupling subset within V localized tightly near the origin boundary $r = 0$.

II. 4TH-ORDER SBP OPERATOR (SBP4.JL)

The module `sbp4.jl` establishes the 4th-order operator utilizing a tightly banded vector-mass modification block near the physical boundaries.

A. Interior Stencil

For the vast majority of the interior grid (nodes strictly unaffected by the boundary perturbation matrices), the localized divergence exactly replicates the standardized

4th-order Cartesian central difference stencil. Evaluated at a node x_i with grid spacing h , the finite-difference operator G_{full} acts as:

$$(G_{\text{full}}u)_i = \frac{1}{12h} (u_{i-2} - 8u_{i-1} + 8u_{i+1} - u_{i+2}) + \mathcal{O}(h^4) \quad (5)$$

B. Boundary Closure and Variable Point Modifications

Because P must be non-trivially manipulated to satisfy the formal energy bounds, boundary nodes $i \in [1, 5]$ are logically relaxed into free numerical algebraic symbols.

In `sbp4.jl`, the scalar mass S remains diagonal but holds 5 free parameters for boundary modification: $s_1, \dots, s_5$. For indices $i \geq 6$, the scalar masses identically match the unperturbed metric transformation $s_i = (H_{\text{cart_half}} r^p)_i$.

The vector mass matrix V carries a localized symmetric tridiagonal-style block near the origin for indices up to 5:

- Fixed Origin: $V_{1,1} = 1$
- Diagonal Unknowns: $V_{2,2}, V_{3,3}, V_{4,4}, V_{5,5}$
- Off-Diagonal Unknown Pairs: $V_{2,3} = V_{3,2}, V_{3,4} = V_{4,3}, V_{4,5} = V_{5,4}$

For indices greater than 5, the diagonal of V directly conforms to S , meaning $V_{i,i} = s_i \forall i \geq 6$, and strictly isolates boundary cross-coupling.

C. Programmatic Operator Imposition

Rather than hardcoding arbitrary floating-point boundary coefficients, the Julia implementation executes an exact rational linear solver (`sbp4_solve_accuracy_constraints`) solving $Ax = b$ populated by constraint polynomials:

1. **Uniform Linear Gradients:** $D \cdot r = (p+1)$ mapping uniform slopes across $N - 4$ nodes.
2. **Polynomial Derivative Moments:** $D \cdot r^3 = (p+3)r^2$ imposed physically over the first 5 rows (`rows_r3 = collect(1:min(5, N))`).
3. **Global Mass Integral Accuracy:** $1^T S 1 = \frac{R^{p+1}}{p+1}$ tracking quadrature sum invariants.

III. 6TH-ORDER SBP OPERATOR (SBP6.JL)

The secondary module `sbp6.jl` natively incorporates the substantially broader stencil sampling geometry requisite to force global 6th-order error scaling, $\mathcal{O}(h^6)$, dictating an exceptionally expansive boundary perturbation layer.

A. Interior Stencil Configuration

The interior folded spatial gradient G_{even} maps precisely to the 6th-order interior central difference derivative projected into rational matrices:

$$(G_{\text{full}}u)_i = \frac{-u_{i-3} + 9u_{i-2} - 45u_{i-1} + 45u_{i+1} - 9u_{i+2} + u_{i+3}}{60h} \quad (6)$$

B. Expanded Boundary Closure Framework

To critically counterbalance the mathematically dominant $u_{i \pm 3}$ nodes propagating beyond the coordinate origin boundaries, the degrees of algebraic freedom inside the V boundary sub-matrix are heavily expanded (as defined via `sbp6_v_offdiag_pairs(N)`).

- Diagonal perturbation parameters now range through $V_{2,2}$ to $V_{10,10}$.
- The localized boundary block transitions functionally to a **pentadiagonal bandwidth pattern**. For nodes $i \in [2, 5]$, interactions natively cross-track radial coupling distances of 1 and 2: $(i, i+1)$ and $(i, i+2)$. Examples include $(2, 3)$ and $(2, 4)$, up through $(5, 7)$. Row 6 supports only an edge-capping pair $(6, 7)$.

C. Constraints and Augmented Methodology

Given the vastly extended stencil limits, `sbp6.jl` introduces significant programmatic complexity regarding constraint mapping to retain positive-definiteness natively. To construct stable bounds mapped across 6th-order global interpolations, the script aggressively extends structural polynomial invariants:

```
add_degree_constraints!(5, rows.rows_r5, :divergence)
```

This systematically imposes $D \cdot r^5 = (p+5)r^4$ against the first m origin nodes. By enforcing highly erratic, non-linear algebraic derivative limits, the code generates stable pentadiagonal boundaries retaining exact mathematical precision under finite limits. Moreover, all N scalar diagonal masses within S are temporarily freed within the constraint domain during the sparse factorization process to preserve matrix compliance limits.

IV. ALGORITHMIC METHODOLOGY AND OPTIMIZATION

A. Matrix Formulation with Exact Relational Arithmetic

The central optimized innovation within these modules lies specifically in circumventing generic IEEE-754

64-bit floating point approximations. Classical inverted boundaries theoretically drift from exact symmetry by factors exceeding 10^{-16} , degrading eigenvalue limits over extended high-frequency time integration iterations.

```
Rq = _sbp4_as_big_rational(R)
```

Both `sbp4.jl` and `sbp6.jl` natively promote all geometrical factors, constraints, and difference geometries into absolute `Rational{BigInt}` components.

B. Parity Folding and Symmetrical Base Gradients

Instead of deriving incredibly complex, numerically ill-conditioned one-sided differences at boundary nodes, the framework implements an elegant Parity Folding scheme. The code evaluates exact symmetrical operations scaling from $-R$ to R , capturing boundary parity via analytical parity matrices E_{even} (for even base functions) and E_{odd} (for odd nodes).

```
Geven = sparse(Rop * Gfull * Eeven)
```

This functionally projects origin conditions out to symmetrical analytical geometry, guaranteeing boundary symmetry inherently rather than iteratively.

C. Matrix-Based Optimization Assembly

All algebraic manipulations resolve perfectly analytically via linear solvers against parameter subsets, and uniquely, are populated *only subsequently* into normalized Sparse arrays utilizing standard library constructors:

```
S = spdiags(0 => sdiag)
I, J, GV = findnz(sparse(Geven))
Gq = sparse(...)
```

This separates arbitrary theoretical scaling logic against high-performance runtime evaluations, rendering matrix-free scaling entirely sparse and bounded, ideal for rapid computational PDE integrators running large radial models structurally dependent on stable properties.